

Machine Learning

Dr. Irfan Yousuf

Department of Computer Science (New Campus)

UET, Lahore

(Week 16; December 29, 2025 - January 02, 2026)

Outline

- Generative Adversarial Networks (GANs)

Generative vs. Discriminative Models

- **Generative models:** These models focus on understanding how the data is generated.
- They aim to learn the distribution of the data itself.
- For instance, if we're looking at pictures of cats and dogs, a generative model would try to understand what makes a cat look like a cat and a dog look like a dog. It would then be able to generate new images that resemble either cats or dogs.

Generative vs. Discriminative Models

- **Discriminative models:** These models, on the other hand, focus on **distinguishing** between different types of data.
- They don't necessarily learn or understand how the data is generated; instead, they learn the boundaries that separate one class of data from another.
- Using the same example of cats and dogs, a discriminative model would learn to tell the difference between the two, but it wouldn't necessarily be able to generate a new image of a cat or dog on its own.

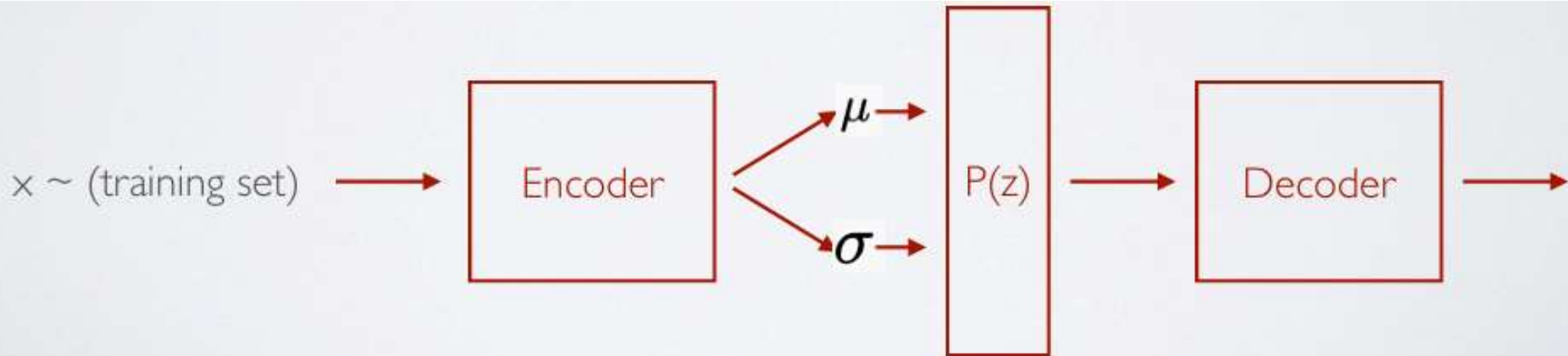
Generative Adversarial Networks

Generative Adversarial Networks

Generative Models

We try to learn the underlying the distribution from which our dataset comes from.

Eg: Variational AutoEncoders (VAE)



Generative Adversarial Networks

Generative Adversarial Networks



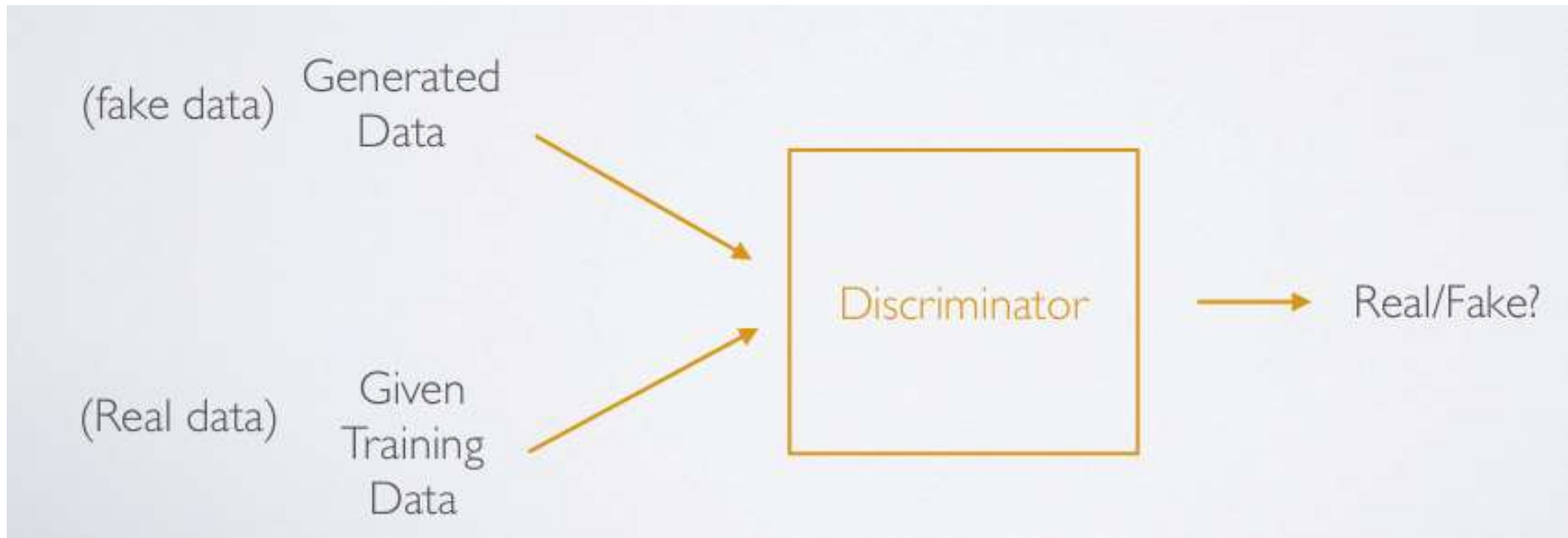
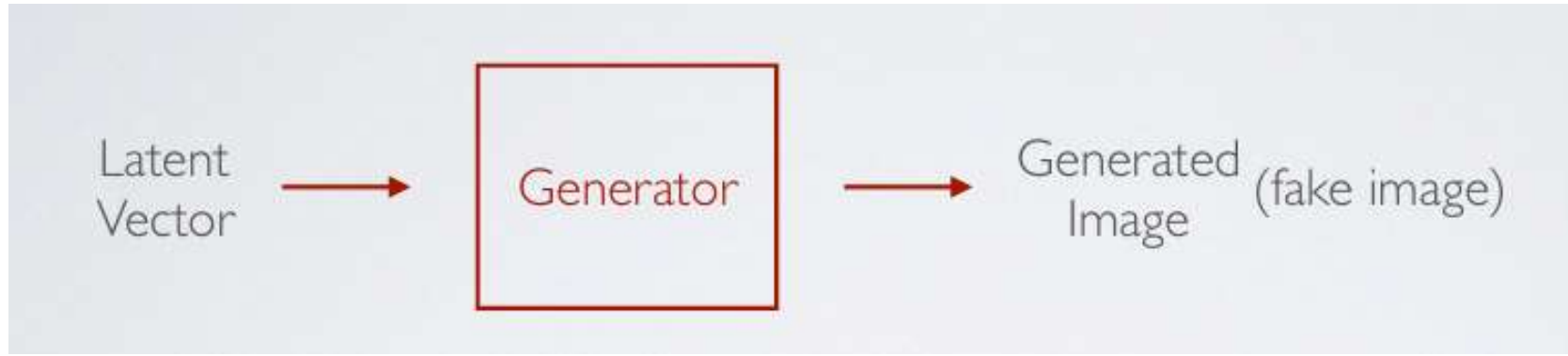
Generative Models

We try to learn the underlying the distribution from which our dataset comes from.
Eg: Variational AutoEncoders (VAE)

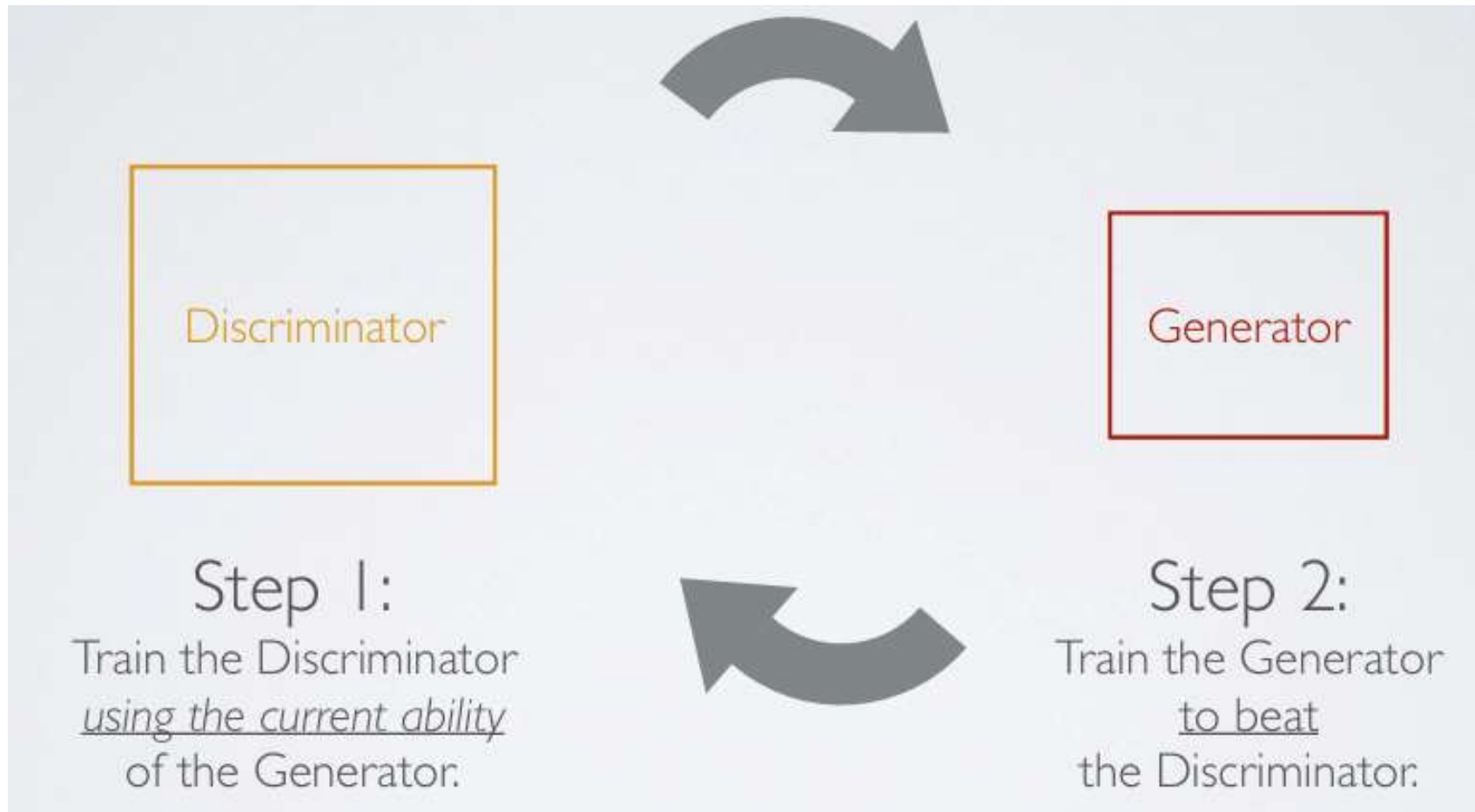
Adversarial Training

GANS are made up of two competing networks (adversaries) that are trying beat each other.

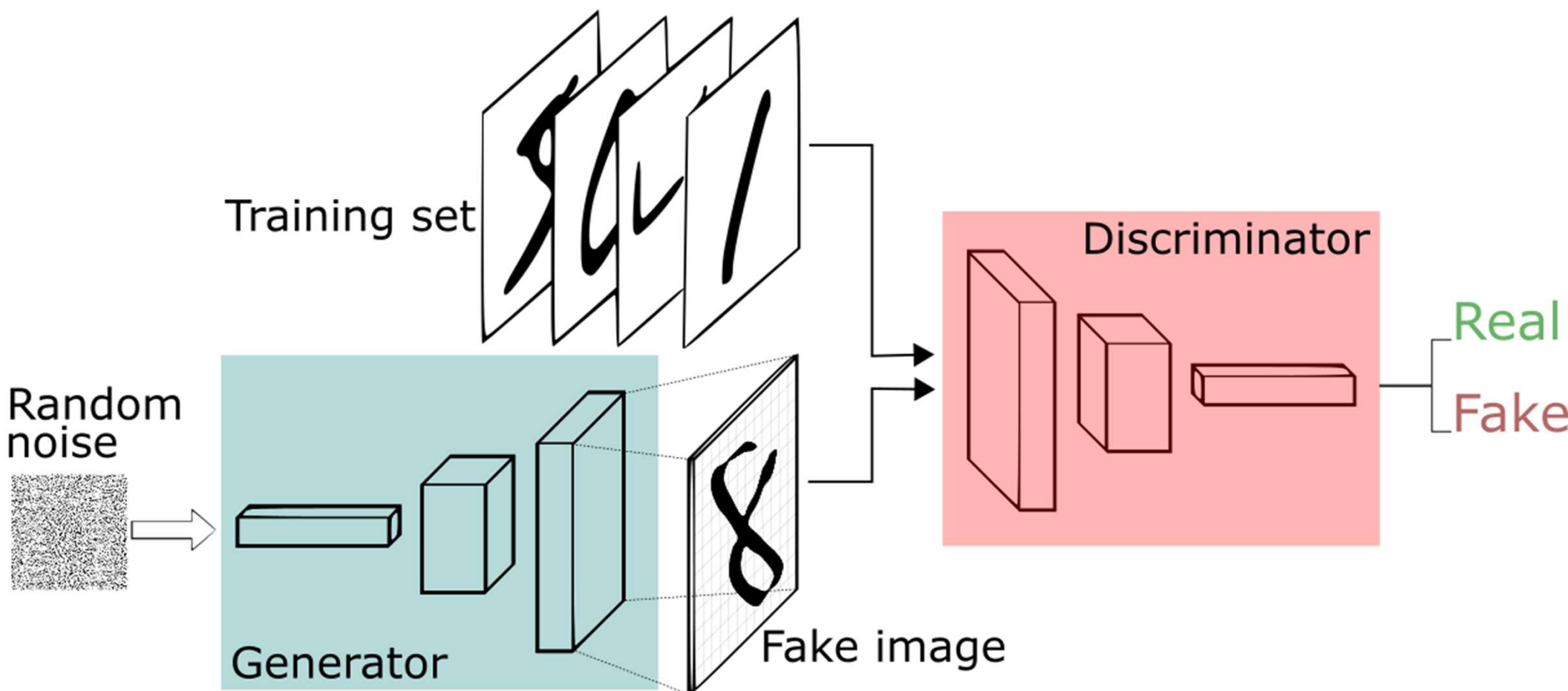
Generative Adversarial Networks



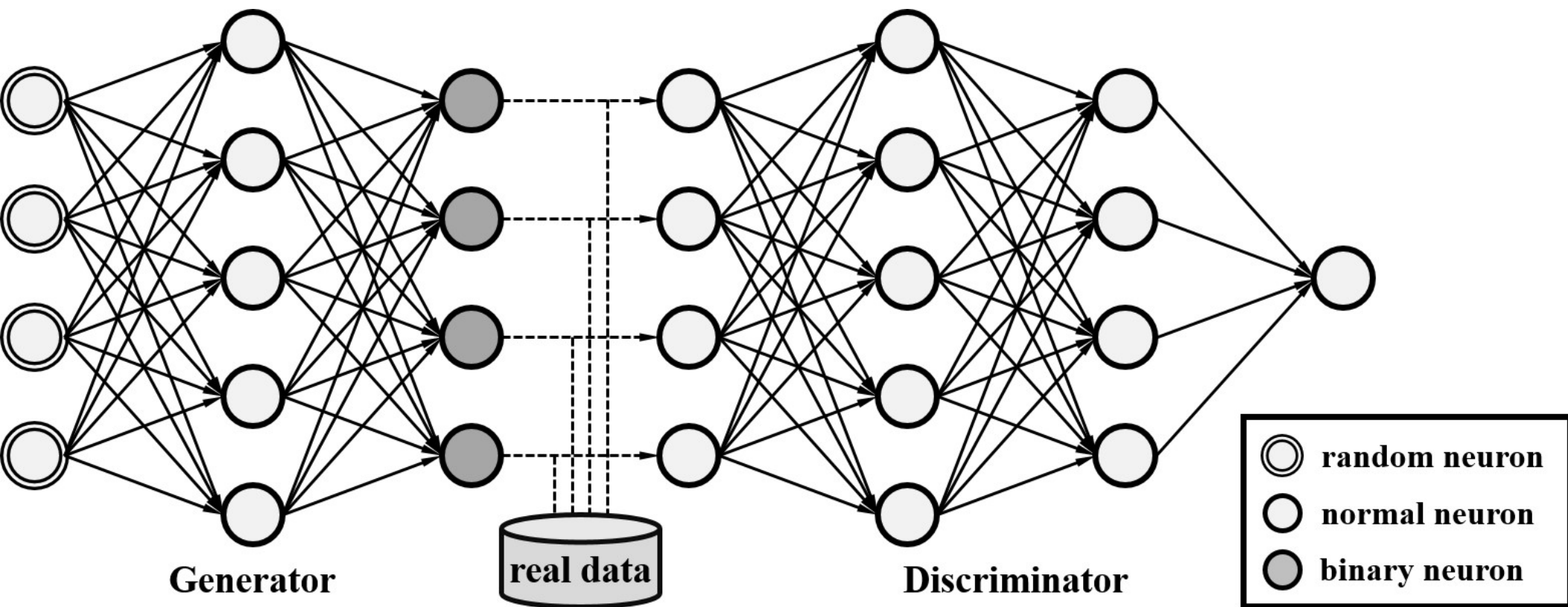
Generative Adversarial Networks



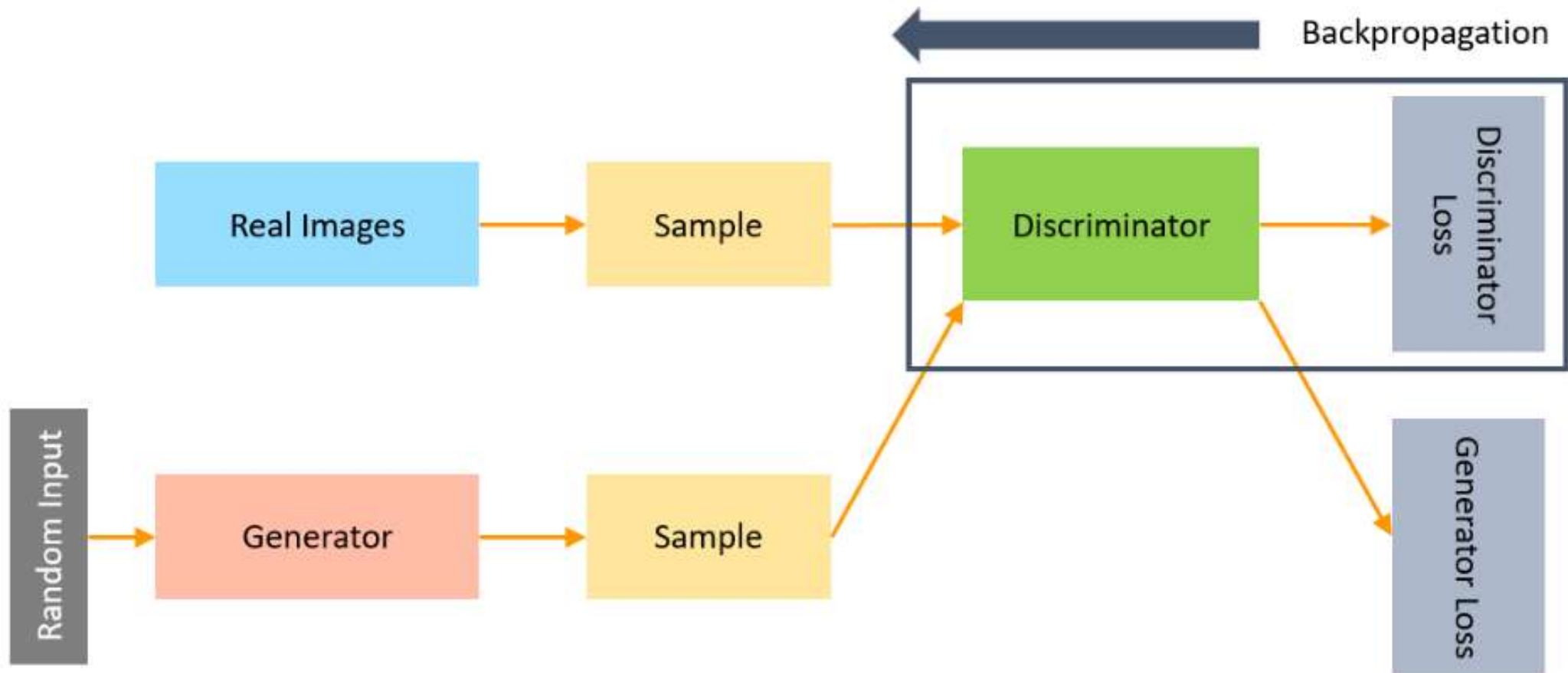
Generative Adversarial Networks



Generative Adversarial Networks



GANs Architecture



GANs Loss Function

The **cross entropy** between two distributions, which we'll call p and q , is defined as

$$H(p, q) := - \sum_i p_i \log q_i$$

where p and q denote a “true” and an “empirical/estimated” distribution, respectively.

To apply this loss function to the current binary classification task, for (again) a *single data point* (x_i, y_i) , we define its “true” distribution as $\mathbb{P}[y_i = 0] = 1$ if $y_i = 0$, or $\mathbb{P}[y_i = 1] = 1$ if $y_i = 1$. Putting in 2-D vector form, it's either $[1, 0]$ or $[0, 1]$. Intuitively, we know for sure which class this belongs to (because it's part of the training data), so it makes sense for a probability distribution to be a “one-hot” vector.

Thus, for one data point x_1 and its label, we get the following loss function, where here I've changed the input to be more precise:

$$H((x_1, y_1), D) = -y_1 \log D(x_1) - (1 - y_1) \log(1 - D(x_1))$$

GANs Loss Function

In the case of GANs, we can say a little more about what these terms mean. In particular, our x_i s only come from two sources: either $x_i \sim p_{\text{data}}$, the true data distribution, or $x_i = G(z)$ where $z \sim p_{\text{generator}}$, the generator's distribution, based on some input code z .

In addition, we also want exactly half of the data to come from these two sources.

To apply this to the sum above, we need to encode this probabilistically, so we replace the sums with expectations, the y_i labels with $1/2$, and we can furthermore replace the $\log(1 - D(x_i))$ term with $\log(1 - D(G(z)))$ under some sampled code z for the generator. We get

$$H((x_i, y_i)_{i=1}^{\infty}, D) = -\frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} \left[\log D(x) \right] - \frac{1}{2} \mathbb{E}_z \left[\log(1 - D(G(z))) \right]$$

GANs Loss Function

- The original GAN Loss function, also known as Min-Max Loss, is a fundamental concept in GANs. It serves as the “scorecard” for both the Generator and Discriminator, guiding their training process through a competitive dynamic.

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

- The generator tries to minimize this function while the discriminator tries to maximize it.

GANs Loss Function

- $\min_G$: This indicates that we are minimizing the loss function with respect to the Generator.
- $\max_D$: This signifies that we are maximizing the loss function with respect to the Discriminator (D).
- $V(D, G)$: It tells us about the overall loss function that depends on both the generator and the Discriminator.

GANs Loss Function

- E : This denotes the expectation operator, which essentially calculates the average value of a function across the entire data distribution.
- In simpler terms, it considers all the possible data points and their probabilities to arrive at a representative value.
- E_x : This denotes the expectation over the real data distribution (represented by x).
- E_z : This signifies the expectation over the random noise distribution (represented by z).

GANs Loss Function

- $\log D(x)$: This term represents the Discriminator's loss function when presented with real data(x).
- Ideally, the Discriminator should output a value close to 1 (indicating “real”) for real data.
- Therefore, high log value (log of a value close to 1) translates to a low loss for the discriminator which it wants to maximize.
 - $\text{Log}(1) = 0$
 - $\text{Log}(0.9) = -0.15$
 - $\text{Log}(0.5) = -1$
 - $\text{Log}(0.1) = -3.3$

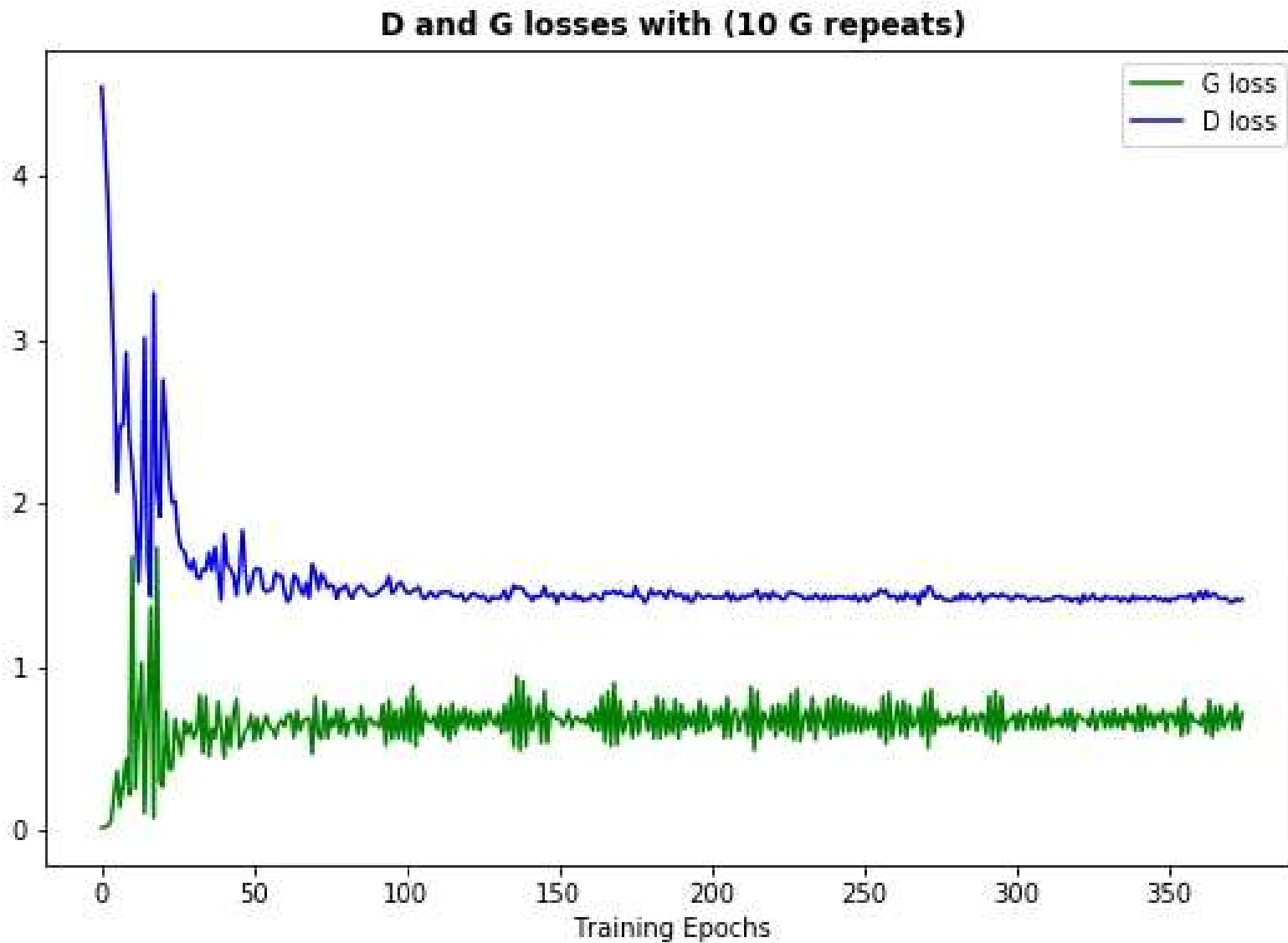
GANs Loss Function

- $\log(1-D(G(z)))$: This term represents the Generator's loss.
- It calculates the logarithm of (1 minus the Discriminator Output) when presented with generated data ($G(z)$).
- The Discriminator should ideally output a value close to 0 (indicating “Fake”) for generated data.
- Therefore, high log value (logarithm of a value close to 0) translates to a high loss for the generator, which it wants to minimize.

GANs Loss Function: The Ideal Scenario:

- The minmax loss strives for a state of balanced competition: The Generator's loss should be neither too high nor too low. A very high loss indicates the Discriminator is easily spotting fakes, while a very low loss suggests the generator isn't challenged enough.
- The Discriminator's loss should also be in a sweet spot. A very low loss signifies the Discriminator might be stuck, while a very high loss might indicate training instability.

GANs loss



Discriminator Loss

- While the discriminator is trained, it classifies both the real data and the fake data from the generator.
- It penalizes itself for misclassifying a real instance as fake, or a fake instance (created by the generator) as real, by maximizing the below function.

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D \left(\mathbf{x}^{(i)} \right) + \log \left(1 - D \left(G \left(\mathbf{z}^{(i)} \right) \right) \right) \right]$$

- $\log(D(\mathbf{x}))$ refers to the probability that the discriminator is rightly classifying the real image,
- maximizing $\log(1-D(G(\mathbf{z})))$ would help it to correctly label the fake image that comes from the generator.

Discriminator Loss

- Correct Classification ($\log D(x) = \log(0.9) \approx -0.146$): The logarithm of a value close to 1 (confident correct classification) results in a smaller negative value. This contributes less penalty to the Discriminator's overall loss, as it performed well on this data point.
- Incorrect Classification ($\log D(x) = \log(0.1) \approx -3.303$): The logarithm of a value close to 0 (mistaken classification) results in a much larger negative value. This significantly increases the Discriminator's loss, penalizing it for the mistake and encouraging improvement.

Discriminator Loss

- **Key Point:** The logarithm magnifies the penalty for the Discriminator when it makes mistakes, urging it to become more expert at distinguishing real from fake data.

Generator loss

- While the generator is trained, it samples random noise and produces an output from that noise. The output then goes through the discriminator and gets classified as either “Real” or “Fake” based on the ability of the discriminator to tell one from the other.
- The generator loss is then calculated from the discriminator’s classification – it gets rewarded if it successfully fools the discriminator, and gets penalized otherwise.

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D \left(G \left(z^{(i)} \right) \right) \right)$$

Generator loss

- **Function:** The Generator's loss measures how successfully the generated data ($G(z)$) can fool the Discriminator into believing it's real.
- **Logarithm's Role:** Similar to the Discriminator's loss, the logarithm emphasizes the importance of the Discriminator's confidence level in classifying the generated data.

Generator loss

- Fooled by Generator ($\log (1 - D(G(z))) = \log(1-0.9) \approx -0.146$): The logarithm of a value close to 0 (successful deception) results in a smaller negative value. This contributes less penalty to the Generator's loss, indicating it achieved its goal of creating realistic data.
- Not Fooled by Generator ($\log (1 - D(G(z))) = \log(1-0.1) \approx -3.303$): The logarithm of a value close to 1 (failed deception) results in a much larger negative value. This significantly increases the Generator's loss, penalizing it for creating unrealistic data and prompting it to improve its generation techniques.

Generator loss

- **Key Point:** The logarithm emphasizes the importance of the Discriminator being thoroughly fooled by the generated data. A larger negative loss signifies the Generator needs to refine its creations.

Role of Log in GANs loss

- Logarithms serve a crucial purpose in the Min-Max loss function by:
- Highlighting even small improvements or discrepancies in the Discriminator's and Generator's performance.
- Encouraging the Discriminator to become a more discerning critic.
- Motivating the Generator to create data that is increasingly difficult for the Discriminator to classify as fake.

GANs loss

- **If the Generator's Loss is too High:** This means the Discriminator is easily spotting the fake output by generator. The Generator needs to improve the skills to create more convincing data.
- **If the Discriminator's Loss is too High:** This means that Generator is constantly fooling the Discriminator. However, this doesn't necessarily guarantee the generated data is truly realistic. The Discriminator need to sharpen its skills to effectively distinguish forgeries.

Types of GANs

Types of GANs

- Vanilla GAN
- Conditional Gan (CGAN)
- Deep Convolutional GAN (DCGAN)
- CycleGAN
- Generative Adversarial Text to Image Synthesis
- Style GAN
- Super Resolution GAN (SRGAN)

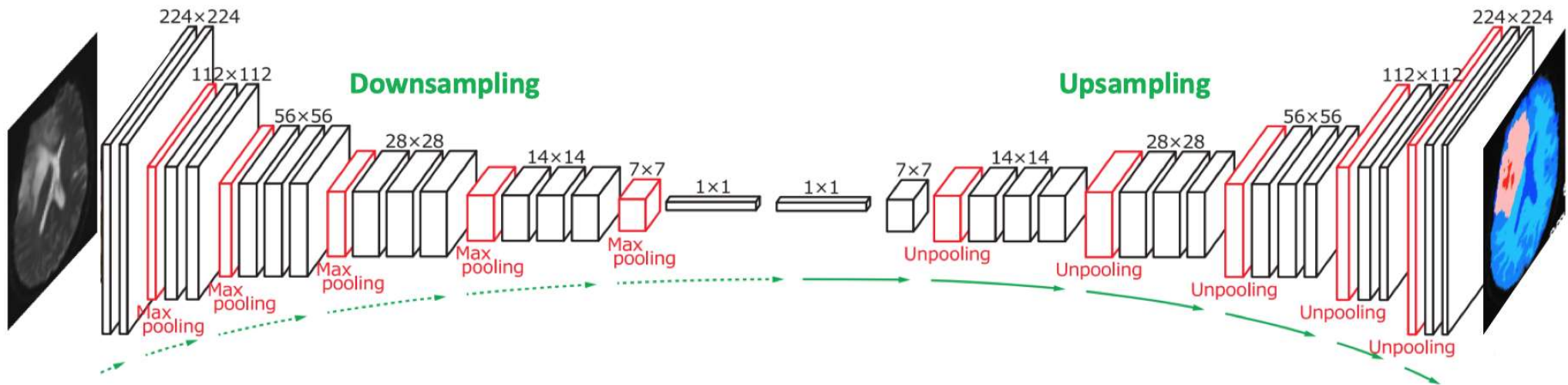
Strided vs. Fractionally-strided Convolution

- In a standard convolution operation, a filter (also called a kernel) is applied to the input data, and the result is a weighted sum of the values in the input region covered by the filter. This operation **reduces** the spatial dimensions of the input.
- Fractionally-strided convolution is designed to perform the opposite operation, i.e., **increase** the spatial dimensions.

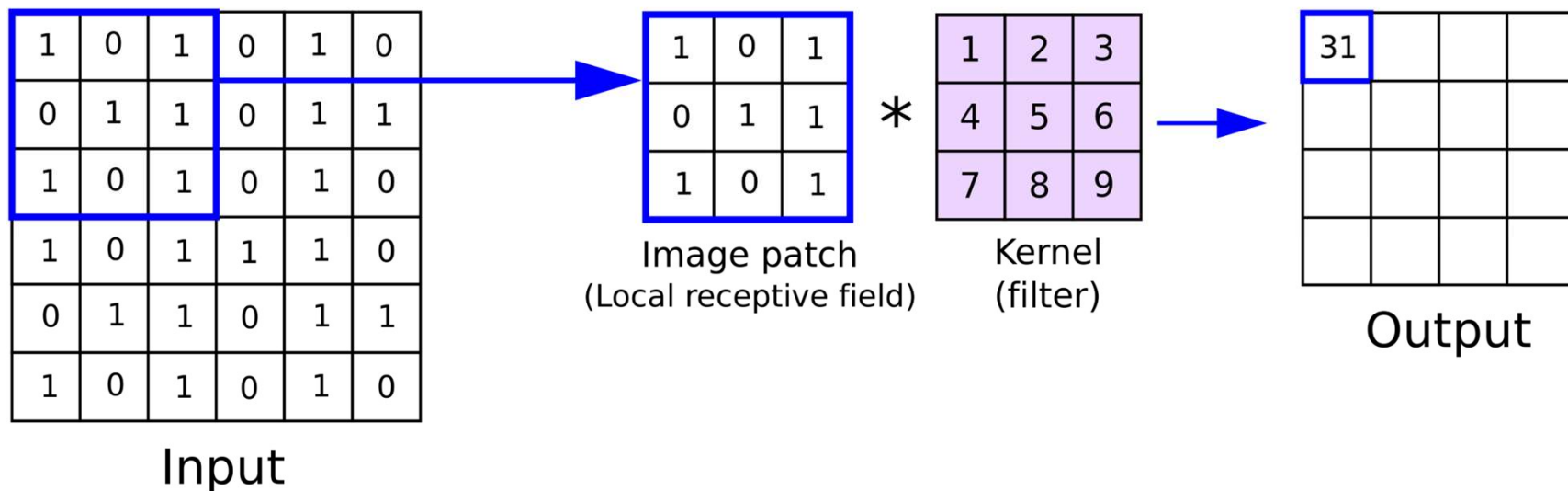
Strided vs. Fractionally-strided Convolution

- The standard convolutional operation reduces the spatial dimensions (width and height) of the input and intermediate feature maps, a process known as **encoding or downsampling**.
- The fractionally-strided convolution gradually restores the spatial dimensions of the feature maps while reducing the number of channels. This process is known as **upsampling or decoding**.

Strided vs. Fractionally-strided Convolution



Strided Convolution (Downsampling)



$$\text{Output size} = \left(\frac{n + 2p - f}{s} + 1 \right) \times \left(\frac{n + 2p - f}{s} + 1 \right)$$

where :

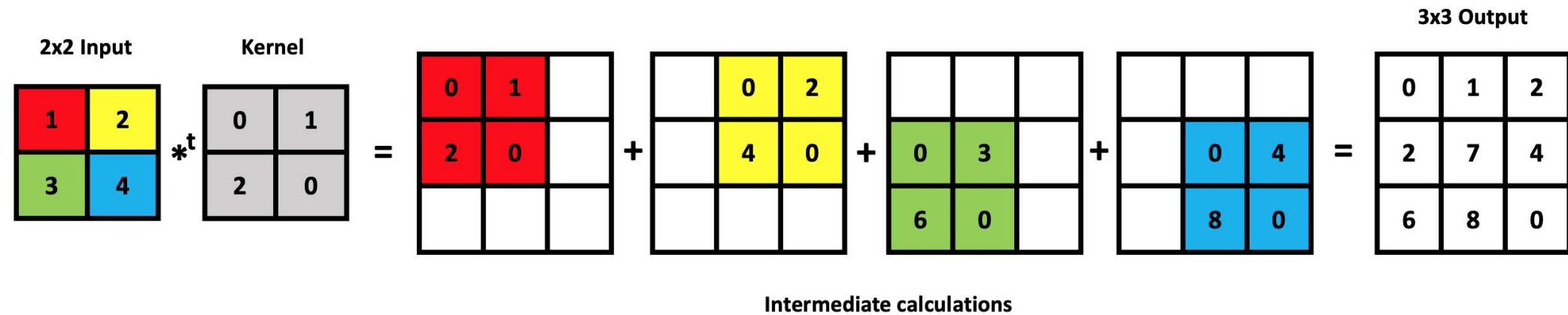
n = input height or width

p = padding

s = stride

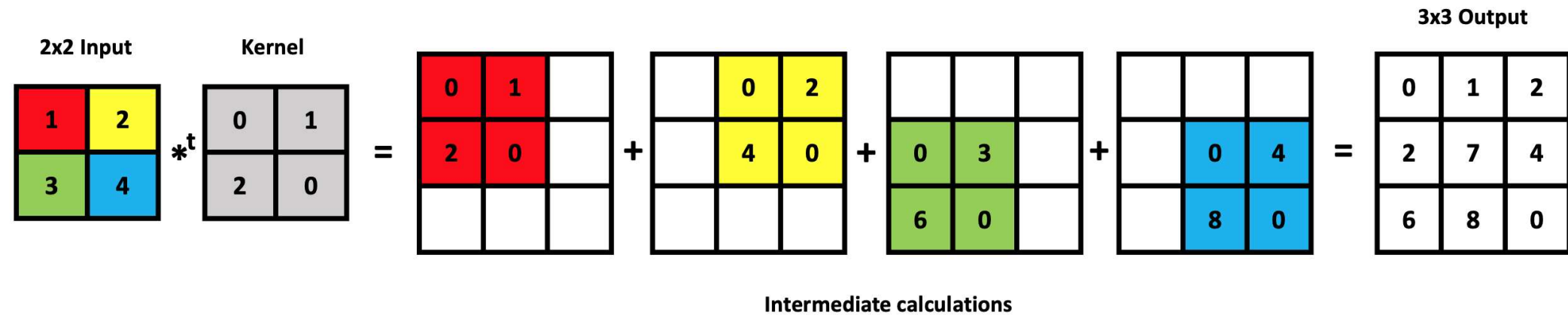
f = filter size

Fractionally-Strided Convolution (Upsampling)



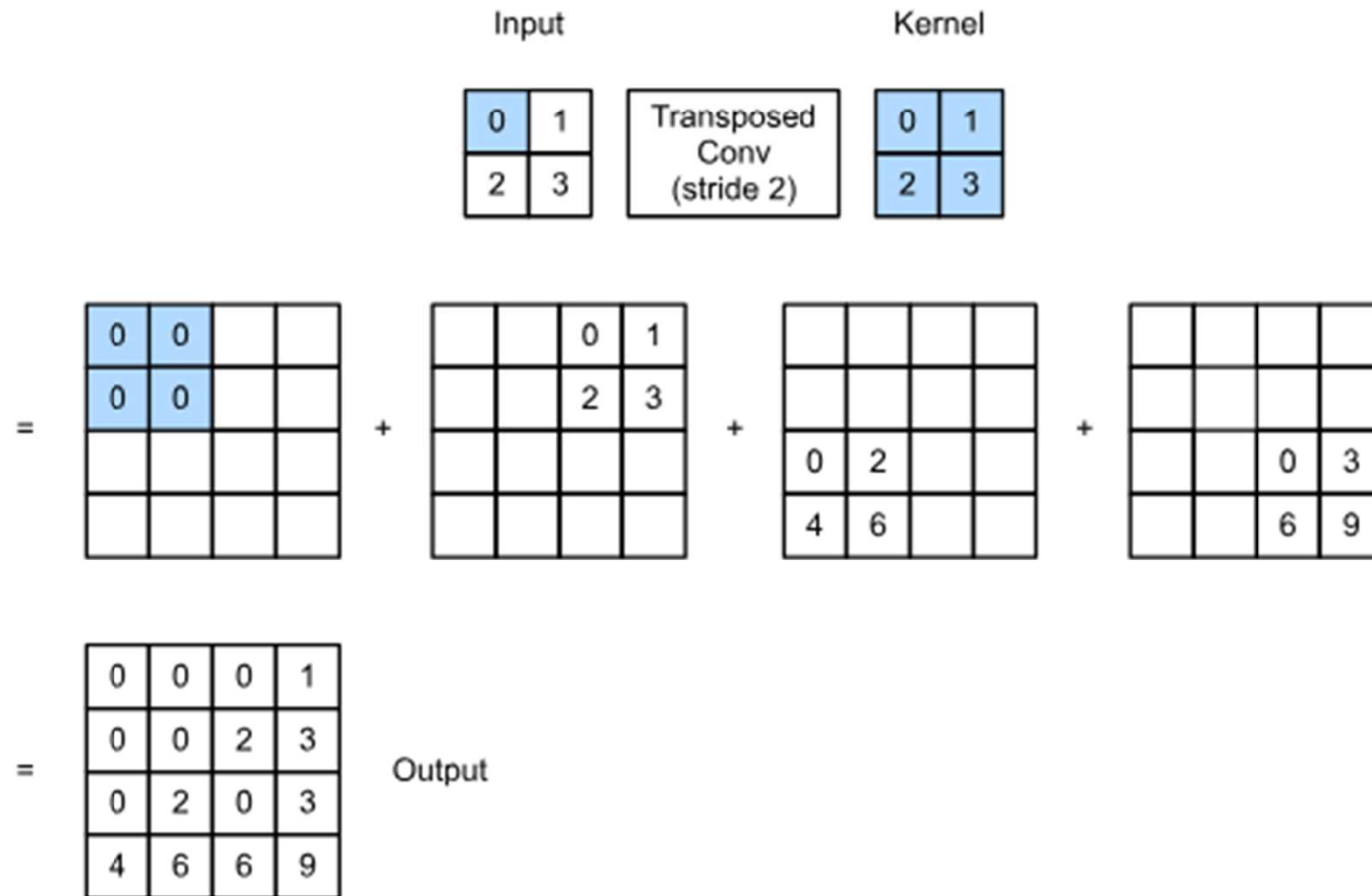
$$\text{output size} = (\text{input size} - 1) * \text{stride} - 2 * \text{padding} + (\text{kernel size} - 1) + 1$$

Fractionally-Strided Convolution (Upsampling)



Transposed Convolution of 2x2 input with a 2x2 kernel to produce a 3x3 output. At each step, a single element (e.g., red) of the input matrix multiplies all 4 kernel elements, with the result placed in an intermediate 3x3 matrix at a location corresponding to position of the red input element. The process is repeated for the three other input elements. The four intermediate matrices are added together to create the final 3x3 output.

Fractionally-Strided Convolution (Upsampling)



$$\text{output size} = (\text{input size} - 1) * \text{stride} - 2 * \text{padding} + (\text{kernel size} - 1) + 1$$

Deep Convolutional GAN (DCGAN)

- DCGAN, or Deep Convolutional GAN, is a generative adversarial network architecture. It uses a couple of guidelines, in particular:
- Replacing any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- Using batchnorm in both the generator and the discriminator.
- Removing fully connected hidden layers for deeper architectures.
- Using ReLU activation in generator for all layers except for the output, which uses tanh.
- Using LeakyReLU activation in the discriminator for all layer.

Deep Convolutional GAN (DCGAN)

UNSUPERVISED REPRESENTATION LEARNING WITH DEEP CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORKS

Alec Radford & Luke Metz

indico Research

Boston, MA

`{alec, luke}@indico.io`

Soumith Chintala

Facebook AI Research

New York, NY

`soumith@fb.com`

Deep Convolutional GAN (DCGAN)

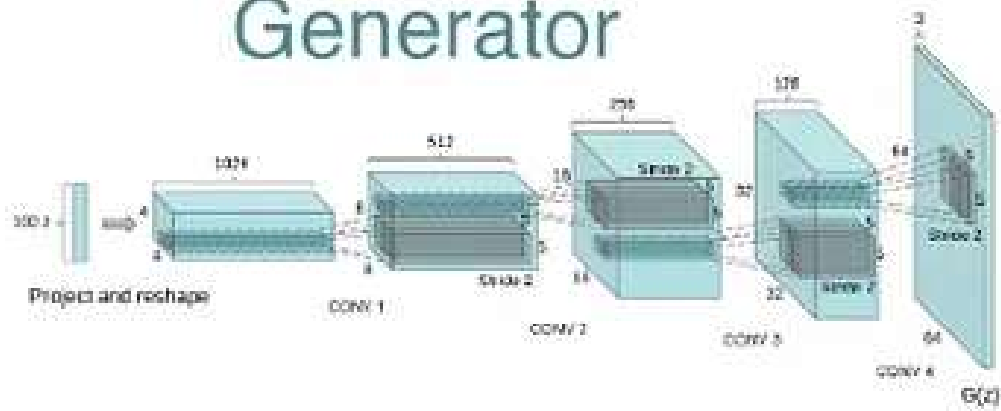
- In recent years, supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications.
- Comparatively, unsupervised learning with CNNs has received less attention.
- We introduce a class of CNNs called deep convolutional generative adversarial networks (DCGANs), that have certain architectural constraints, and demonstrate that they are a strong candidate for unsupervised learning.

DCGAN Architecture

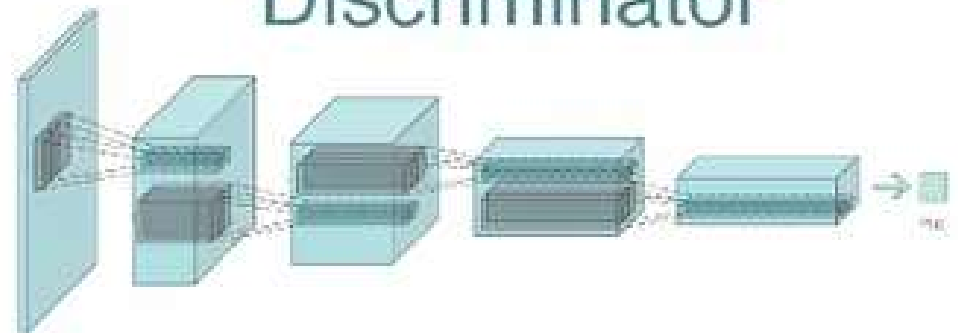
- DCGAN uses convolutional and convolutional-transpose layers in the generator and discriminator, respectively.
- The discriminator consists of strided convolution layers, batch normalization layers, and LeakyRelu as activation function. It takes a 3x64x64 input image.
- The generator consists of convolutional-transpose layers, batch normalization layers, and ReLU activations. The output will be a 3x64x64 RGB image.

DCGAN Architecture

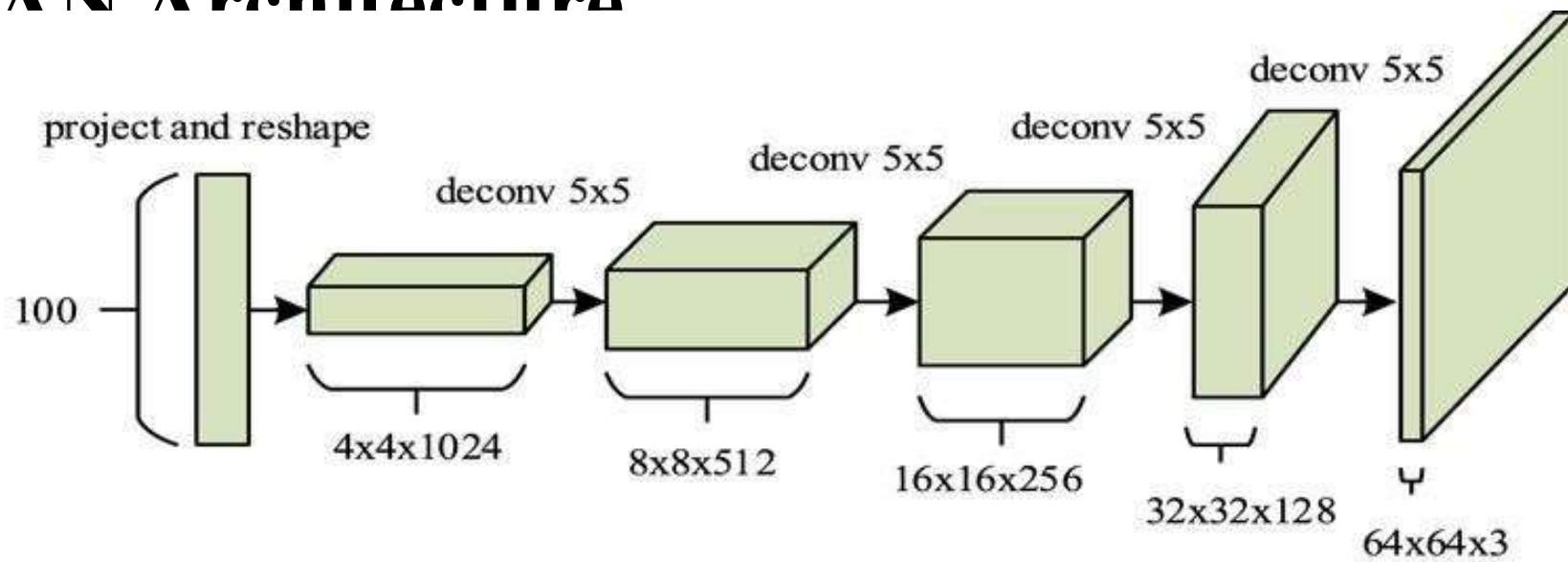
Generator



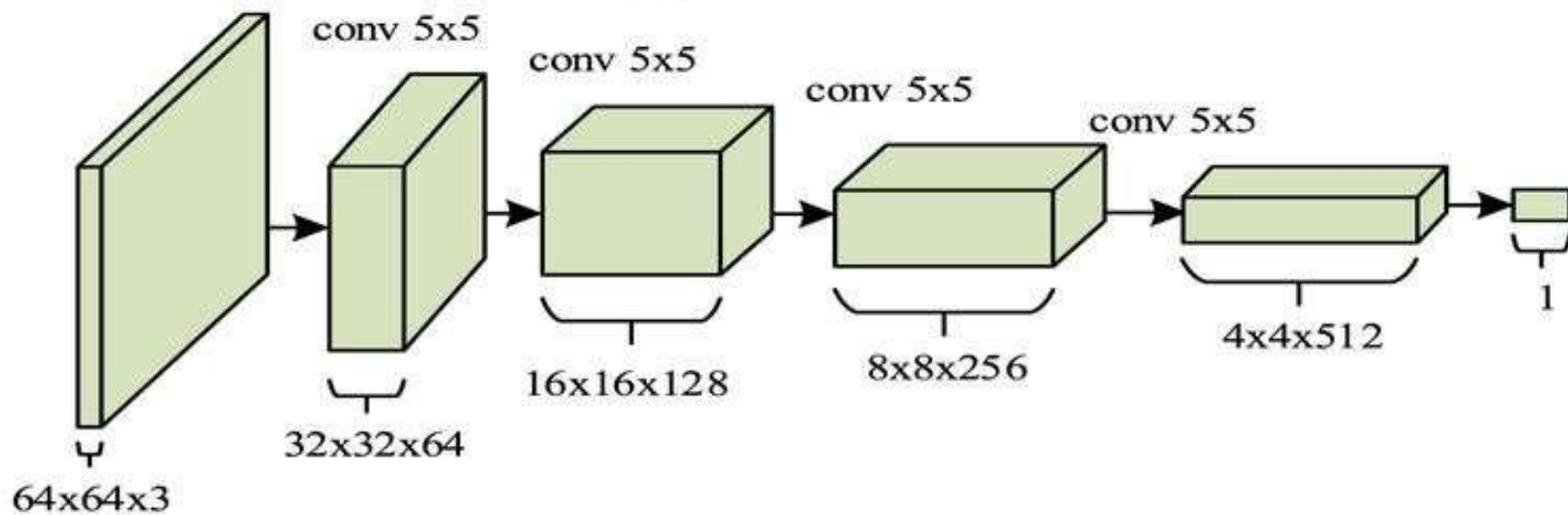
Discriminator



DCGAN Architecture



(a) Generator



(b) Discriminator

DCGAN Architecture: Generator

- The DCGAN generator is designed to synthesize realistic images from random noise.
- It typically comprises several layers of transposed convolutions, also known as deconvolutions or upsampling layers.
- Starting with a random noise vector as input, often drawn from a standard normal distribution, this vector is projected through a fully connected layer to a higher-dimensional space.

DCGAN Architecture: Generator

- The subsequent layers use transposed convolutions to progressively upsample the features, gradually transforming the initial noise into a complex image representation.
- Batch normalization, introduced in each layer, helps stabilize the learning process by normalizing the input to each layer and reducing internal covariate shifts

DCGAN Architecture: Discriminator

- In contrast, the discriminator network within a DCGAN functions as a binary classifier, distinguishing between real and generated images.
- It comprises a series of convolutional layers, each followed by batch normalization and activation functions like LeakyReLU to introduce non-linearity.
- Strided convolutions are employed to reduce the spatial dimensions of the input, gradually extracting high-level features to discern real images from synthetic ones.

DCGAN Architecture: Discriminator

- The discriminator's architecture does not include fully connected layers, aligning with the principles of convolutional networks to capture spatial hierarchies effectively.
- The discriminator ends with a sigmoid activation function, producing a probability score indicating the authenticity of the input image.

DCGAN generated images



DCGAN generated images



Generative Adversarial Text to Image Synthesis

Generative Adversarial Text to Image Synthesis

Scott Reed, Zeynep Akata, Xincheng Yan, Lajanugen Logeswaran
Bernt Schiele, Honglak Lee

REEDSCOT¹, AKATA², XCYAN¹, LLAJAN¹
SCHIELE², HONGLAK¹

¹ University of Michigan, Ann Arbor, MI, USA (UMICH.EDU)

² Max Planck Institute for Informatics, Saarbrücken, Germany (MPI-INF.MPG.DE)

Generative Adversarial Text to Image Synthesis

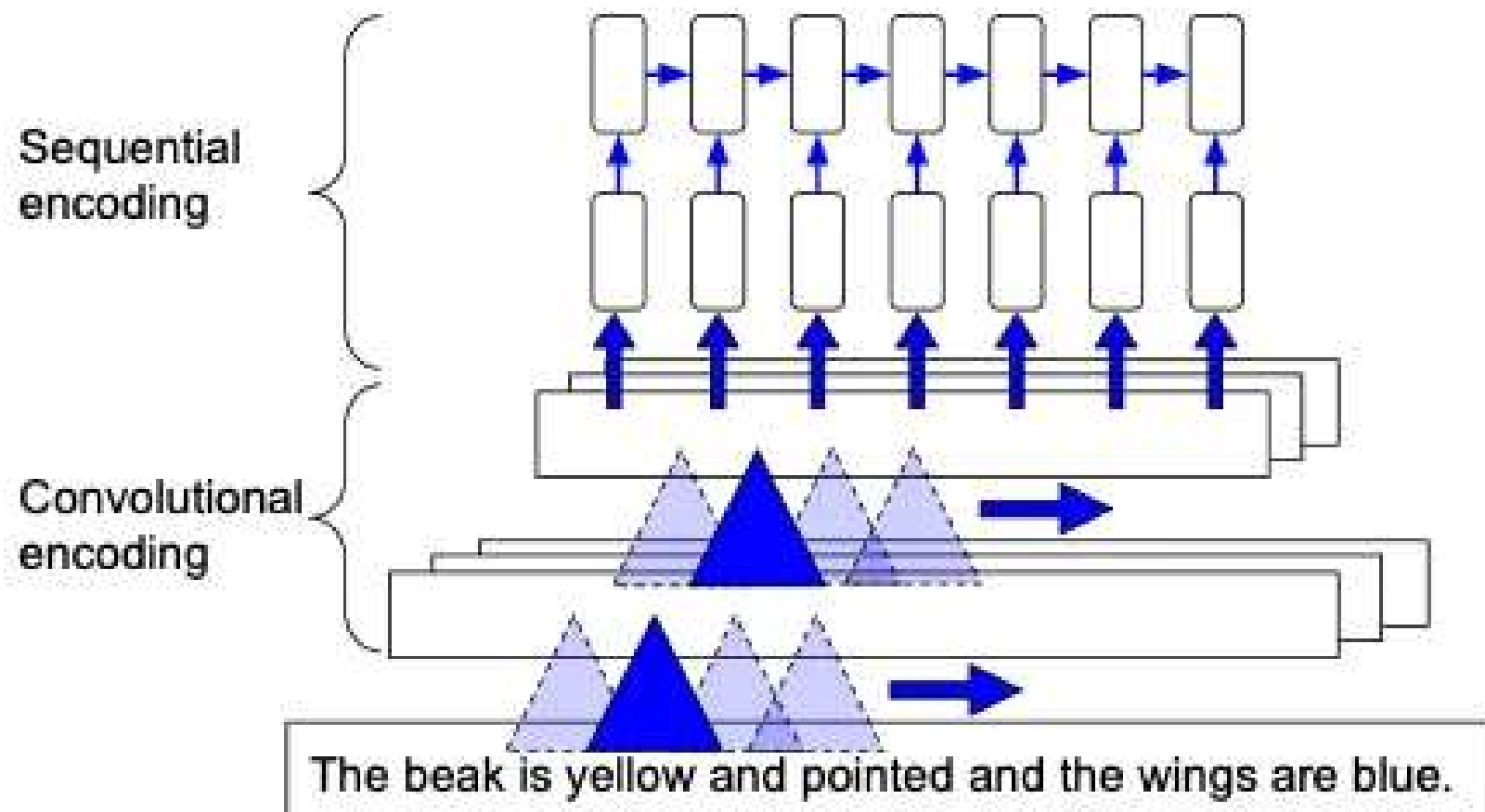
- Automatic synthesis of realistic images from text would be interesting and useful, but current AI systems are still far from this goal.
- However, in recent years generic and powerful recurrent neural network architectures have been developed to learn discriminative text feature representations. Meanwhile, deep convolutional generative adversarial networks (GANs) have begun to generate highly compelling images of specific categories, such as faces, album covers, and room interiors.

Generative Adversarial Text to Image Synthesis

- In this work, we develop a novel deep architecture and GAN formulation to effectively bridge these advances in text and image modeling, translating visual concepts from characters to pixels.
- We demonstrate the capability of our model to generate plausible images of birds and flowers from detailed text descriptions.

Model Architecture: RNN

- Use of RNN to obtain the deep symmetric structured joint embeddings.



Model Architecture: RNN

- The authors first use a RNN stacked on-top of a CNN in order to obtain encoded features of sentence descriptions (the process of obtaining vectors).
- The combination of these two types of models is used in order to get the “best of both worlds”.
- Because CNN is fast and scalable, it is used to learn low-level features efficiently.
- RNN is more expensive, but also far better at learning more abstract features.

Model Architecture: RNN

- So, the authors encode the sentences using the above hybrid text encoder and they encode the images using a CNN.
- Now, with these two representations, they minimize the below loss.
- Here, v refers to visual information and t refers to text descriptions. Therefore, the objective is symmetric with respect to both images and text.

$$\frac{1}{N} \sum_{n=1}^N \Delta(y_n, f_v(v_n)) + \Delta(y_n, f_t(t_n)) \quad (1)$$

Model Architecture: GAN

- Use of deep convolutional GANs, which have traditionally been used for image generation.
- They repurpose the DC-GAN architecture by conditioning the GANs on the text feature representation obtained in the first sub-process.

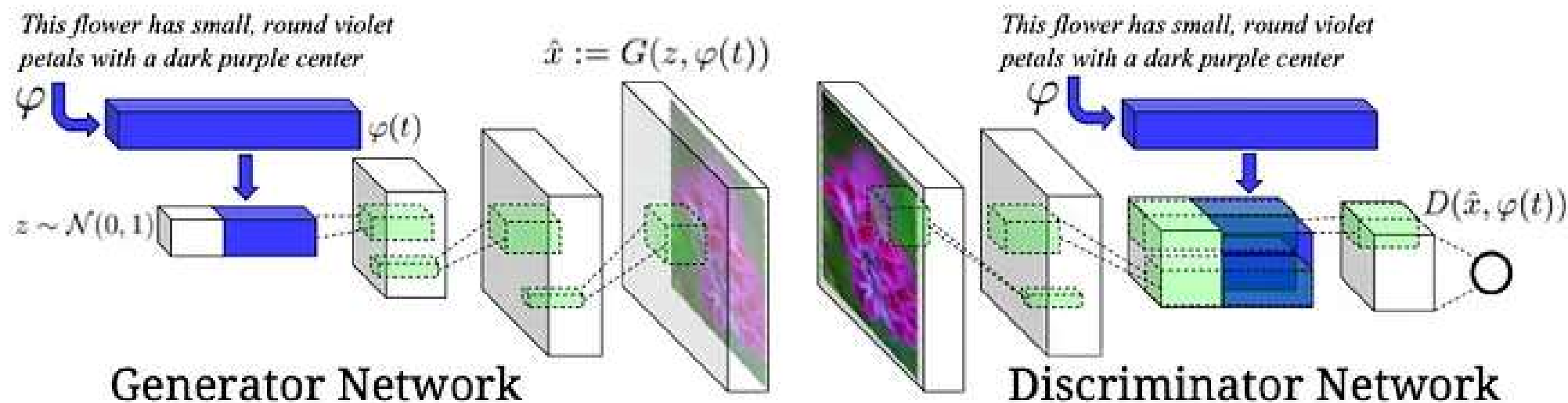


Figure 2. Our text-conditional convolutional GAN architecture. Text encoding $\varphi(t)$ is used by both generator and discriminator. It is projected to a lower-dimensions and depth concatenated with image feature maps for further stages of convolutional processing.

Model Architecture: GAN

- The text description is encoded, compressed to a lower dimension using a fully connected layer with leaky Relu, and concatenated to noise vector (The noise vector is used to sample random properties, referred to as “style” from the image space).
- From there, an image is generated using standard feedforward inference (deconvolution). In other words, the generator network.

Model Architecture: GAN

- The synthetic image is processed by the discriminator, which uses stride 2 convolution, spatial batch norm, leaky relu.
- The encoded text description is used again when it is concatenated with the outputs of convolution in the discriminator network.

Qualitative Analysis

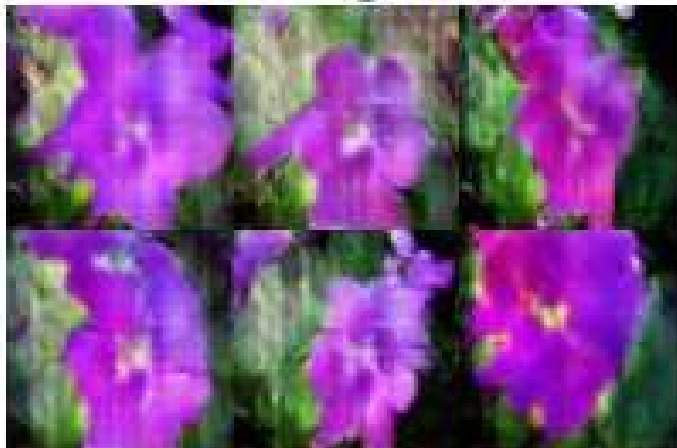
this small bird has a pink breast and crown, and black primaries and secondaries.



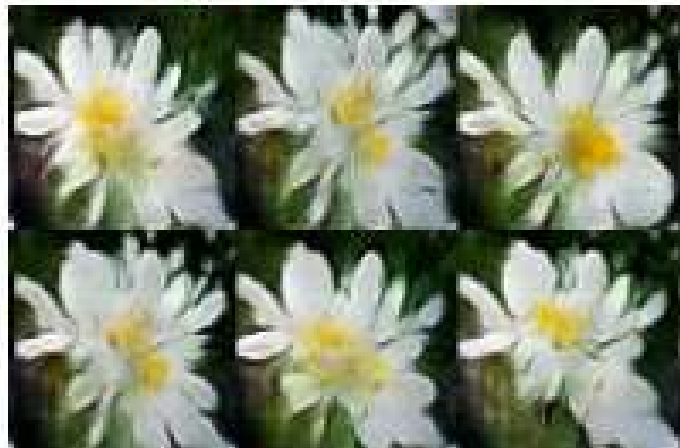
this magnificent fellow is almost all black with a red crest, and white cheek patch.



the flower has petals that are bright pinkish purple with white stigma



this white and yellow flower have thin white petals and a round yellow stamen



Introduction to GenAI

Generative AI

- Generative Artificial Intelligence (Generative AI) refers to AI systems that can create new content rather than just analyze or classify existing data.
- What it can generate:
 - Text (essays, summaries, chat responses)
 - Images (art, photos)
 - Audio (music, speech)
 - Code (programs, scripts)
 - Video
- **How it works (basic idea):** Generative AI learns patterns from large amounts of data and then uses those patterns to produce new, similar content.

Large Language Models

- A Large Language Model (LLM) is a type of generative AI that is specialized for understanding and generating human language.
- **Key characteristics:**
 - Trained on massive text datasets (books, articles, websites, etc.)
 - Uses probability to predict the next word in a sequence
 - Can answer questions, summarize text, translate languages, and write code
- Examples: GPT, BERT, Claude, LLaMA

Key Terms

- A **model** is a mathematical system that learns patterns from data and makes predictions or generates output.
- **Training** is the process of showing the model large amounts of data so it can learn relationships between words, sentences, or concepts.
- **Parameters** are internal values the model learns during training.
- More parameters → generally more expressive power
- LLMs often have billions of parameters

Key Terms

- **Parameters** are the internal numbers a neural network learns during training.
 - They are mostly weights and biases in the model's layers
 - Each parameter influences how strongly one piece of information affects another
 - During training, these values are adjusted to reduce prediction errors
- So, a model with 1 billion parameters has 1,000,000,000 adjustable numbers that together encode patterns about language: grammar, facts, reasoning strategies, styles, etc.

Key Terms

- Small model (millions of parameters): learns basic patterns
- Large model (billions of parameters): can represent much more complex relationships, abstractions, and knowledge.
- More parameters $\neq$ automatically better, but they increase potential capability when paired with enough data and good training.

Key Terms

Why does parameter count matter?

Parameter count is a rough proxy for:

1. **Expressive power** – how complex the patterns the model can represent
2. **Memory** – how much information it can store implicitly
3. **Compute cost** – more parameters $\rightarrow$ more GPU/TPU memory and computation
4. **Training data needs** – large models require enormous datasets to avoid overfitting

Key Terms

LLMs don't read text as words, but as **tokens**.

- A token may be a word, part of a word, or punctuation
- Example:

“unbelievable” → ["un", "believ", "able"] (approx.)

Models predict the **next token**, not the next word.

The vocabulary is the set of all tokens the model knows.

- Typical size: 30,000–100,000 tokens
- Each token has a learned vector representation

Key Terms

Most modern LLMs are based on the **Transformer** architecture.

Key components:

- **Self-attention** — lets the model decide which tokens matter most
- **Feed-forward layers** — process information at each position
- **Stacked layers** — deeper layers learn higher-level abstractions

Key Terms

Attention determines how much each token should focus on other tokens.

Example:

“The animal didn’t cross the street because it was tired.”

Attention helps the model infer that “it” = the animal, not the street.

Key Terms

LLMs are usually trained in two phases:

1. **Pretraining**

- Task: predict the next token
- Data: massive text corpora (books, websites, code, etc.)

2. **Fine-tuning / alignment**

- Task: follow instructions, be helpful, safe, and accurate
- Often uses human feedback

Key Terms

The context window is how much text the model can “see” at once.

- Measured in tokens (e.g., 4k, 32k, 128k tokens)
- Longer context = better long-term coherence, but higher cost

Summary

- Generative Adversarial Networks